

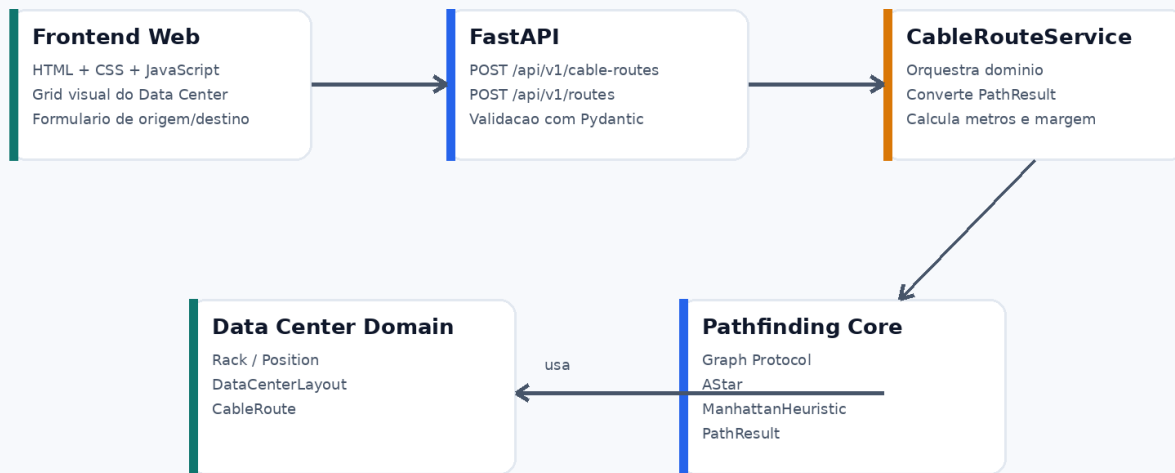
# RACKROUTE API

## Data Center Cable Router + Reusable Pathfinding Core

### Documento de formalizacao do MVP

#### RackRoute - Arquitetura do MVP

Separacao entre apresentacao, aplicacao, dominio e nucleo algoritmico



*Escopo: primeiro projeto publicavel no GitHub, com backend REST e frontend visual minimo.*

#### Objetivo do MVP

Construir uma API em Python capaz de calcular rotas fisicas de cabeamento entre racks de um Data Center, evitando regioes bloqueadas e exibindo o resultado em uma interface web simples. O algoritmo A\* fica isolado em um nucleo reutilizavel.

Versao 0.1 - Projeto de portfolio

# 1. Visao do produto

RackRoute e um pequeno sistema backend-first para planejamento de rotas de cabeamento em Data Centers. A aplicacao modela a planta fisica como um grafo em grid, usa A\* para buscar uma rota entre dois racks e traduz o resultado algoritmico em informacoes de dominio, como distancia fisica e comprimento recomendado de cabo.

## Problema

Dado um rack de origem, um rack de destino e um conjunto de regioes onde o cabo nao pode passar, encontrar uma rota valida e eficiente e informar quanto cabo deve ser reservado para a instalacao.

## 1.1 Valor demonstrado no portfolio

- Python aplicado a um problema de infraestrutura, e nao apenas a um exercicio isolado de algoritmo.
- API REST com FastAPI e contratos de entrada/saida claros.
- Separacao entre dominio, aplicacao e mecanismo de pathfinding.
- Uso de grafos, priority queue, A\* Search e heuristica Manhattan.
- Testes automatizados para o nucleo algoritmico e para as regras de negocio.
- Frontend minimo capaz de tornar o comportamento do backend visivel e demonstravel.

## 1.2 Base conceitual do material de referencia

O material "Introduction to A\*", de Amit Patel, fornece a base conceitual usada aqui: pathfinding permite planejar antecipadamente em vez de reagir somente quando um obstaculo e encontrado; problemas de pathfinding podem ser representados como grafos; e A\* combina o custo acumulado  $g(n)$  com uma estimativa heuristica  $h(n)$ , avaliando  $f(n) = g(n) + h(n)$ .

## Nota de escopo

A heuristica Manhattan e a arquitetura de software proposta neste documento sao decisoes de implementacao do projeto. Elas nao sao detalhadas na introducao fornecida; foram adicionadas como desenho tecnico para o MVP.

## 1.3 Fora do escopo da v0.1

- Banco de dados persistente.
- Autenticacao e autorizacao.
- Microservices ou mensageria.
- Mapas fisicos reais/3D.
- Roteamento multiponto.
- Custos dinamicos por temperatura ou capacidade de bandeja.
- Frontend com framework SPA.

## 2. Escopo funcional do MVP

### 2.1 Historias de uso

1. Como usuario, quero selecionar um rack de origem e um rack de destino para calcular uma rota de cabeamento.
2. Como usuario, quero visualizar obstaculos, origem, destino e rota em um grid para entender o caminho calculado.
3. Como usuario, quero definir uma margem de seguranca para receber um comprimento recomendado de cabo.
4. Como desenvolvedor, quero reutilizar o mesmo nucleo A\* em outros dominios sem depender de classes de Data Center.
5. Como desenvolvedor, quero validar o comportamento com testes automatizados e exemplos reproduziveis.

### 2.2 Requisitos funcionais

| ID    | Requisito                                             | Prioridade |
|-------|-------------------------------------------------------|------------|
| RF-01 | Carregar uma planta de Data Center a partir de JSON.  | MVP        |
| RF-02 | Identificar racks por ID.                             | MVP        |
| RF-03 | Calcular rota entre dois racks usando A*.             | MVP        |
| RF-04 | Evitar celulas bloqueadas.                            | MVP        |
| RF-05 | Calcular distancia fisica usando o tamanho da celula. | MVP        |
| RF-06 | Aplicar margem percentual ao comprimento recomendado. | MVP        |
| RF-07 | Expor rota generica e rota de cabo via API REST.      | MVP        |
| RF-08 | Renderizar rota no navegador.                         | MVP        |
| RF-09 | Retornar erro claro quando nao houver rota.           | MVP        |

### 2.3 Requisitos nao funcionais

- Python 3.12 ou superior.
- FastAPI para camada HTTP.
- Type hints nas interfaces publicas.
- Dependencia algoritmica apenas da standard library; heapq para a priority queue.
- Pytest para testes.
- Estrutura de projeto simples, legivel e sem overengineering.
- README reproduzivel com comando de instalacao, execucao e testes.

### 3. Arquitetura do MVP

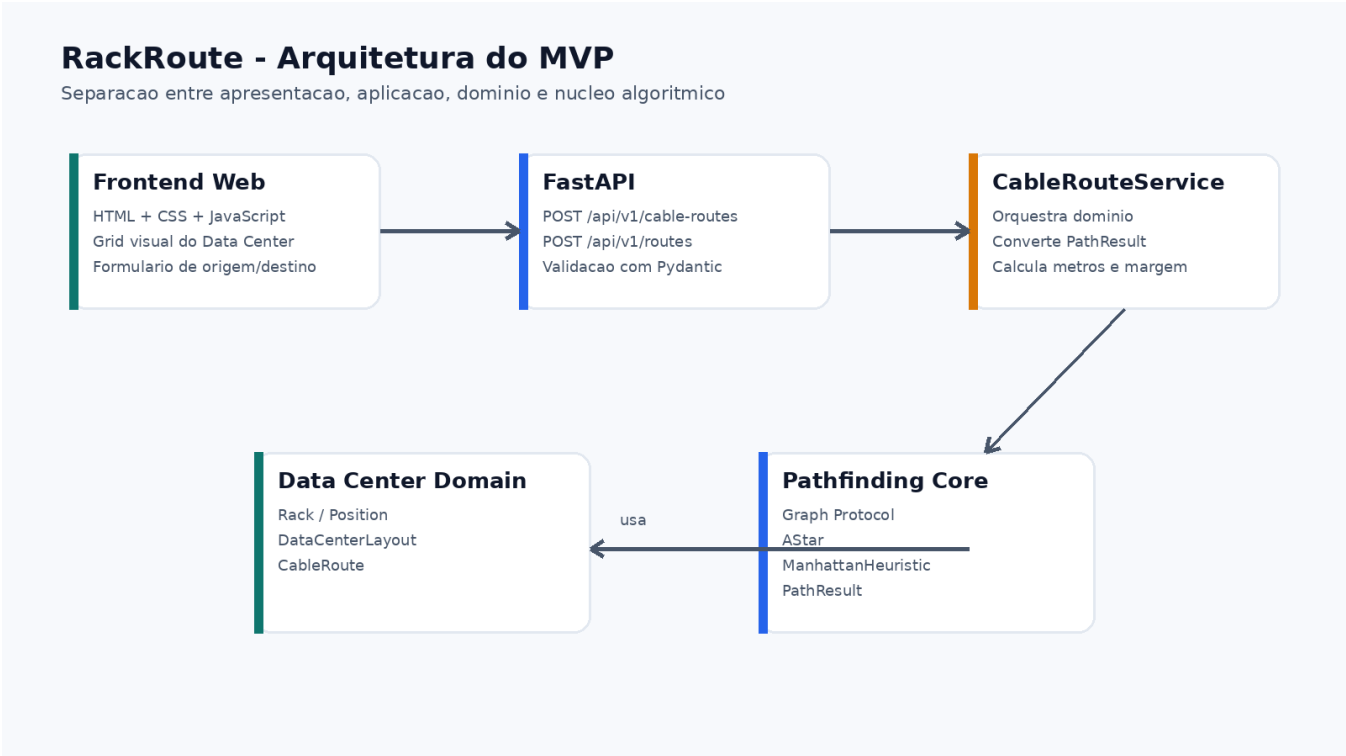


Figura 1 - O frontend consome a API; o dominio usa o nucleo de pathfinding, nunca o contrario.

#### 3.1 Regra principal de dependencia

```
Frontend -> FastAPI -> Application/Domain -> Pathfinding Core
Pathfinding Core -X-> Data Center Domain
```

O A\* deve trabalhar apenas com conceitos genericos de grafo, custo, origem, destino e heuristica. Rack, cabo, metro e layout de Data Center pertencem ao dominio e ficam fora do algoritmo.

#### 3.2 Responsabilidades por camada

| Camada             | Responsabilidade                                                                |
|--------------------|---------------------------------------------------------------------------------|
| Frontend           | Coletar parametros, chamar a API, desenhar o grid e apresentar metricas.        |
| API                | Validar HTTP/JSON, converter schemas e chamar o service.                        |
| Application/Domain | Localizar racks, executar o caso de uso e transformar PathResult em CableRoute. |
| Pathfinding Core   | Executar A* de forma reutilizavel e independente do dominio.                    |
| Infrastructure     | Carregar a planta JSON e demais dados externos.                                 |

## 4. Estrutura sugerida do repositório

```
data-center-cable-router/
|-- src/
|   |-- pathfinding/
|   |   |-- astar.py
|   |   |-- graph.py
|   |   |-- heuristic.py
|   |   |-- manhattan.py
|   |   |-- result.py
|   |-- datacenter/
|   |   |-- models/
|   |   |   |-- position.py
|   |   |   |-- rack.py
|   |   |   |-- layout.py
|   |   |   |-- cable_route.py
|   |   |-- services/
|   |   |   |-- cable_route_service.py
|   |   |-- infrastructure/
|   |   |   |-- layout_loader.py
|   |-- api/
|   |   |-- main.py
|   |   |-- schemas.py
|   |   |-- routes.py
|   |-- web/
|   |   |-- index.html
|   |   |-- app.js
|   |   |-- styles.css
|-- data/
|   |-- example_datacenter.json
|-- tests/
|   |-- pathfinding/
|   |-- datacenter/
|-- README.md
|-- requirements.txt
|-- pyproject.toml
|-- .gitignore
```

### 4.1 Nucleo reutilizavel

O diretorio pathfinding deve poder ser removido do projeto e usado em outro software sem conhecer nenhuma entidade do Data Center. O contrato minimo pode ser expresso por um Graph Protocol com operacoes de vizinhanca e custo, uma heuristica e um AStar.search().

```
class Graph(Protocol):
    def neighbors(self, node): ...
    def cost(self, current, next_node) -> float: ...

result = astar.search(
    graph=layout,
    start=source.position,
    goal=destination.position,
    heuristic=manhattan,
)
```

### 4.2 Heuristica escolhida

Para a v0.1, o grid permite apenas movimentos ortogonais. A heuristica escolhida e a Manhattan Distance:

$$h(n) = |x(n) - x(goal)| + |y(n) - y(goal)|$$

Essa estimativa combina naturalmente com deslocamento horizontal/vertical. Na primeira versao, cada passo valido possui custo unitario.

## 5. Modelo de dominio e dados

### 5.1 Entidades minimas

| Tipo             | Campos essenciais                                                    | Papel                               |
|------------------|----------------------------------------------------------------------|-------------------------------------|
| Position         | x, y                                                                 | Coordenada imutavel do grid.        |
| Rack             | id, position                                                         | Ponto de origem/destino no dominio. |
| DataCenterLayout | width, height, racks, blocked_positions, cell_size_meters            | Implementa o grafo do ambiente.     |
| PathResult       | path, cost, explored_nodes                                           | Resultado puramente algoritmico.    |
| CableRoute       | source, destination, path, distance_meters, recommended_cable_length | Resultado do caso de uso.           |

### 5.2 Arquivo de layout

```
{
  "width": 12,
  "height": 8,
  "cell_size_meters": 0.5,
  "racks": [
    {"id": "RACK-A23", "x": 1, "y": 1},
    {"id": "RACK-D17", "x": 10, "y": 6}
  ],
  "blocked_cells": [[4,1], [4,2], [4,3], [5,3], [6,3]]
}
```

### 5.3 Regra de comprimento do cabo

Se o caminho possui N movimentos e cada celula representa S metros, a distancia e N x S. A recomendacao aplica uma margem percentual configurada pelo usuario.

```
distance_meters = steps * cell_size_meters
recommended_length = distance_meters * (1 + safety_margin)
```

#### Exemplo

32 passos x 0,5 m = 16,0 m. Com margem de 10%, o sistema recomenda 17,6 m de cabo.

## 6. Contrato da API

### 6.1 Endpoint especializado - cable routes

POST /api/v1/cable-routes

Request:

```
{
  "source_rack": "RACK-A23",
  "destination_rack": "RACK-D17",
  "safety_margin": 0.10
}
```

Response:

```
{
  "source": "RACK-A23",
  "destination": "RACK-D17",
  "route": [[1,1], [2,1], [3,1]],
  "steps": 32,
  "distance_meters": 16.0,
  "recommended_cable_length_meters": 17.6,
  "explored_nodes": 48,
  "algorithm": "A*",
  "heuristic": "Manhattan"
}
```

### 6.2 Endpoint generico - reusable route engine

POST /api/v1/routes

Mantem a ideia do RouteFinder API dentro do mesmo repositório. Recebe um grid generico, origem, destino e obstaculos e retorna PathResult. Ele demonstra que o nucleo nao depende do dominio de Data Center.

### 6.3 Erros esperados

- 404 - rack inexistente.
- 400 - margem de seguranca invalida ou request inconsistente.
- 422 - nao existe rota valida entre os racks.
- 200 - origem igual ao destino, com distancia zero.

## 7. Frontend minimo demonstravel

O frontend nao precisa virar um segundo projeto. Para o MVP, a opcao mais enxuta e servir uma pagina HTML/CSS/JavaScript pela propria aplicacao FastAPI. Isso mantem a demo visual, reduz dependencias e deixa a integracao HTTP explicita.

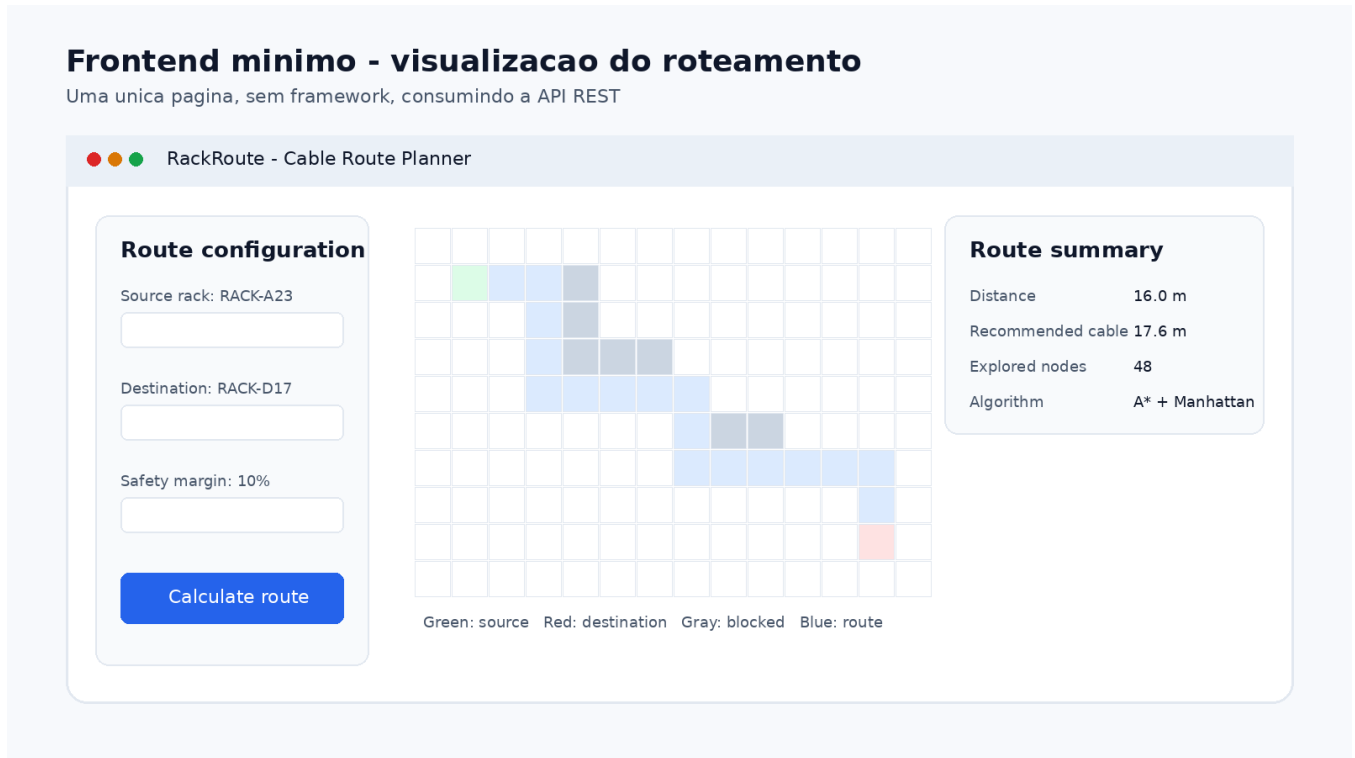


Figura 2 - Wireframe funcional: controles a esquerda, grid no centro e resumo da rota a direita.

### 7.1 Componentes visuais

- Source rack: select preenchido pelos racks do layout.
- Destination rack: select independente para o destino.
- Safety margin: input numerico simples, por exemplo 10%.
- Calculate route: dispara POST /api/v1/cable-routes.
- Grid: representa cada celula da planta e colore origem, destino, bloqueios e rota.
- Route summary: mostra distancia, cabo recomendado, quantidade de nos explorados e algoritmo.

### 7.2 Convencao visual

| Elemento         | Representacao |
|------------------|---------------|
| Origem           | Verde         |
| Destino          | Vermelho      |
| Celula bloqueada | Cinza         |
| Rota calculada   | Azul          |
| Celula livre     | Branco        |

## 7.3 Integracao frontend-backend

```
fetch("/api/v1/cable-routes", {
  method: "POST",
  headers: {"Content-Type": "application/json"},
  body: JSON.stringify(payload)
})
.then(response => response.json())
.then(route => renderRoute(route));
```

O navegador não recalcula o algoritmo. Ele apenas envia a configuração, recebe a lista de coordenadas e pinta as células correspondentes. A regra de negócio continua no backend.

## 8. Fluxo de execucao

### Fluxo de ponta a ponta

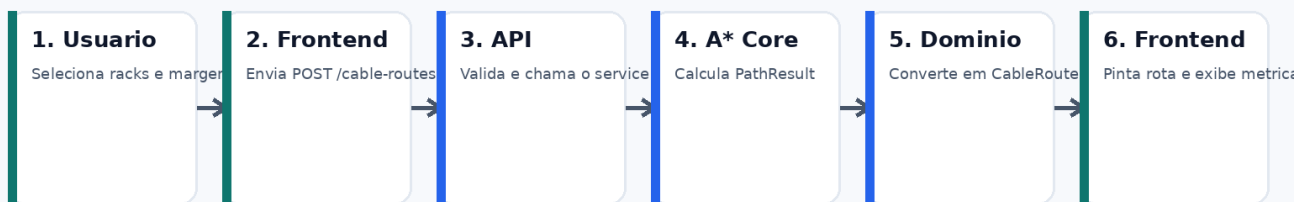


Figura 3 - Da interacao do usuario ate a renderizacao da rota.

6. O usuario abre a pagina e escolhe os racks.
7. JavaScript monta o payload e envia a requisicao HTTP.
8. FastAPI valida os campos e chama CableRouteService.
9. O service localiza os racks no DataCenterLayout.
10. AStar.search() usa Graph + Manhattan para produzir PathResult.
11. O service converte passos em metros, aplica margem e cria CableRoute.
12. A API serializa o resultado.
13. O frontend colore o grid e atualiza o painel de metricas.

### 8.1 Pseudofluxo do caso de uso

```
source = layout.get Rack(source_id)
destination = layout.get Rack(destination_id)
```

```

path_result = astar.search(
    layout,
    source.position,
    destination.position,
    manhattan
)

distance = path_result.steps * layout.cell_size_meters
recommended = distance * (1 + safety_margin)

return CableRoute(...)

```

## 9. Testes e Definition of Done

### 9.1 Casos de teste minimos

- Manhattan retorna a distancia esperada para duas coordenadas.
- A\* encontra uma rota em grid sem obstaculos.
- A\* contorna um bloqueio.
- A\* informa ausencia de rota quando o destino esta isolado.
- Origem igual ao destino retorna custo/distancia zero.
- Layout nao permite atravessar celulas bloqueadas.
- Service calcula metros corretamente.
- Service aplica safety margin corretamente.
- API retorna 404 para rack desconhecido.
- Frontend renderiza a sequencia recebida sem recalculer a rota.

### 9.2 Definition of Done - v0.1

#### O MVP esta pronto quando:

o repositorio instala em ambiente limpo; pytest passa; FastAPI inicia; /docs abre; /api/v1/cable-routes retorna uma rota; a pagina web permite escolher origem/destino e exibe o caminho no grid; README explica arquitetura, execucao e testes.

### 9.3 Comandos esperados

```

python -m venv .venv
# ativar ambiente virtual
pip install -r requirements.txt
pytest
uvicorn src.api.main:app --reload

```

### 9.4 Evidencias para o GitHub

- Screenshot da pagina com uma rota calculada.
- Exemplo de request/response da API.
- Diagrama de arquitetura.

- Resultado dos testes.
- Uma planta de exemplo versionada em data/example\_datacenter.json.

## 10. Plano de construçao do primeiro dia

A ordem abaixo privilegia primeiro o comportamento correto e depois a apresentacao. O frontend entra apenas depois que o caso de uso estiver funcionando via API.

| Etapa | Entrega                                | Checkpoint                              |
|-------|----------------------------------------|-----------------------------------------|
| 1     | Position, Graph, Manhattan, PathResult | Testes unitarios verdes.                |
| 2     | AStar + heapq                          | Rota correta em grid simples.           |
| 3     | Rack + DataCenterLayout + loader JSON  | Layout carregado e vizinhanca validada. |
| 4     | CableRouteService                      | Distancia e margem corretas.            |
| 5     | FastAPI + schemas + endpoints          | Swagger executando requests.            |
| 6     | index.html + app.js + styles.css       | Grid e rota visiveis no browser.        |
| 7     | README + screenshot + limpeza          | Repositorio publicavel.                 |

### 10.1 Corte de escopo se faltar tempo

14. Preservar A\*, dominio, API e testes.
15. Manter apenas um layout JSON fixo.
16. No frontend, usar um grid estatico gerado a partir do layout.
17. Adiar endpoint generico /routes se necessario; o nucleo deve continuar generico internamente.
18. Nunca sacrificar testes basicos para adicionar efeitos visuais.

### 10.2 Extensoes para v0.2+

- Custos diferentes por tipo de celula/bandeja.
- Rotas alternativas.
- Exportacao da rota em JSON/CSV.
- Edicao visual de obstaculos.
- Persistencia de layouts.
- Dockerfile e CI com GitHub Actions.
- Benchmark A\* x Dijkstra.
- Heuristicas adicionais quando o modelo de movimento justificar.

## 11. Resumo tecnico

### Arquitetura em uma frase

Uma interface web minima consome uma API FastAPI; a API chama um caso de uso de Data Center; o caso de uso usa um nucleo A\* desacoplado para calcular a rota; o resultado e convertido em metros e

comprimento recomendado de cabo.

```

Frontend Web
  |
  v
FastAPI
  |
  v
CableRouteService
  |
  +--> DataCenterLayout / Rack / CableRoute
  |
  --> AStar + Graph + Manhattan + PathResult

```

## 11.1 Dependencias sugeridas

```

fastapi
uvicorn
pydantic
pytest
httpx

```

## 11.2 Referencia conceitual

Patel, Amit. "Introduction to A\*", Amit's Thoughts on Pathfinding / Red Blob Games. Material fornecido pelo usuario, atualizado ate 2026. O documento de referencia sustenta a introducao a pathfinding, representacao por grafos, comparacao entre Dijkstra/Greedy e a formulacao de A\* como  $f(n) = g(n) + h(n)$ .

### Deciso es adicionadas neste MVP

Manhattan Distance, organizacao de pastas, FastAPI, frontend HTML/CSS/JavaScript, modelo de dominio do Data Center e regras de calculo de cabo sao escolhas de projeto propostas para transformar os conceitos algoritmicos em um portfolio de Engenharia de Software.

## RACKROUTE API - MVP v0.1

Pronto para implementacao e publicacao inicial no GitHub.